

spring ai alibaba + nacos +mcp 实现mcp服务负载均衡调用实战

程序员越 2026-03-03 494 阅读10分钟

关注

一、项目实现目标

项目目标是完成传统SASS服务向MCP服务的转型，并实现MCP服务的负载均衡调用，具体目标如下：

- **服务转型**：将现有的传统SASS服务快速、高效地封装并转换为MCP（服务，复用原有业务逻辑，降低转型成本。
- **负载均衡**：实现MCP服务的多实例部署与负载均衡调用，通过Nacos服务注册与发现机制，分发请求流量，避免单实例压力过大，提升服务可用性。

文末有源码链接

二、项目运行环境

项目基于Spring生态搭建，依赖以下核心组件，各组件版本及作用明确如下，确保组件间兼容性与功能正常发挥：

- **NACOS 3.1.1**：作为服务注册中心与配置中心，支持MCP服务的注册、发现与配置管理，为MCP服务的负载均衡提供基础支撑，确保服务实例可被正常识别与调用。
- **Spring AI Alibaba**：提供ReactAgent组件，该组件是对Spring AI Alibaba的更高层次封装，集成了Agent开发常用的工具调用、拦截器、内存存储等能力，简化了Agent开发流程。
- **千问大模型（qwen-flash）**：与Spring AI Alibaba深度集成，为Agent提供自然语言交互与逻辑处理能力，支撑控制台的对话式交互。

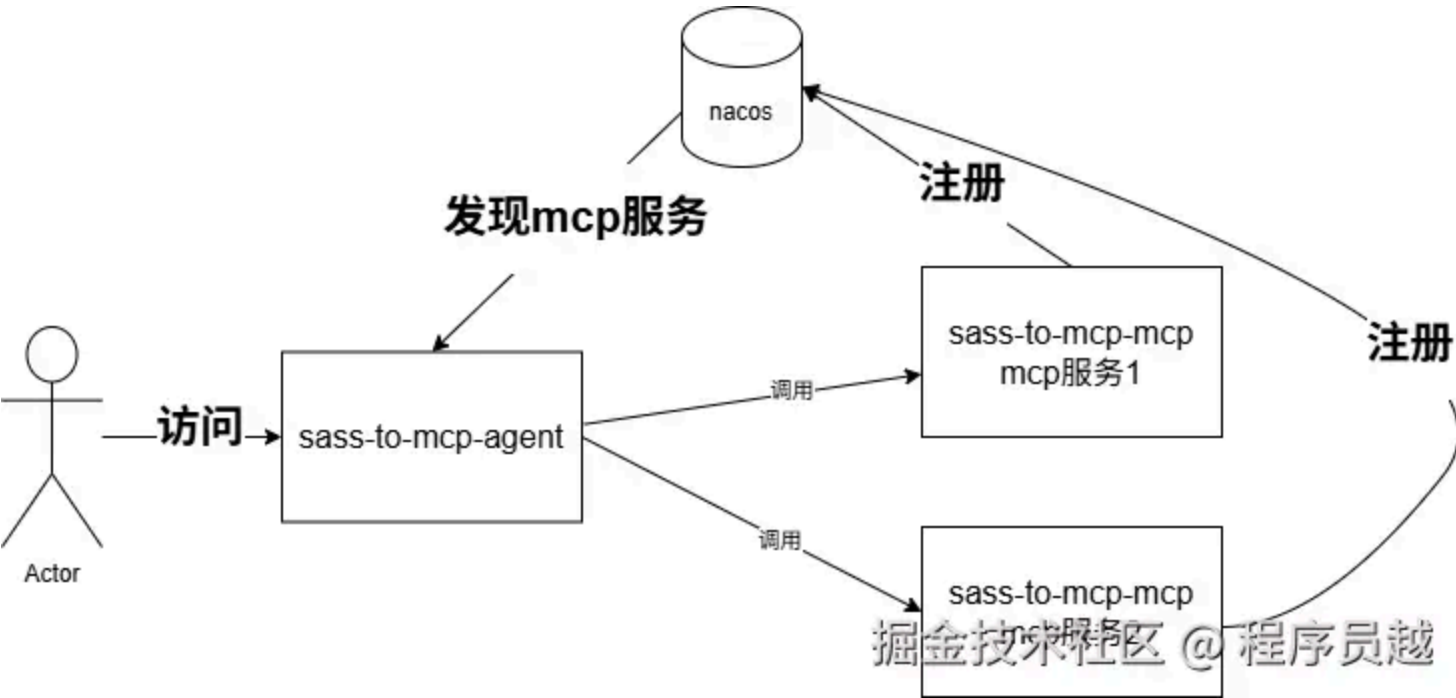


登录掘金领取礼包
更多登录后权益等你解锁

登录 / 注册

Agent

三、项目整体架构



项目采用模块化架构设计，分为两个核心项目模块，整体架构如下：

- **sass-to-mcp-agent**：作为服务调用端，核心功能是通过命令行控制台与用户进行交互，通过Nacos发现已注册的MCP服务，并根据用户输入的指令，自动调用对应的MCP服务工具，返回处理结果。
- **sass-to-mcp-mcp**：作为服务提供端，包含原有SASS服务与封装后的MCP服务，部署时需启动两个实例并注册到Nacos，主要用于模拟MCP服务的多实例部署，配合完成负载均衡测试，验证负载分发效果。

两个模块通过Nacos实现服务注册与发现，Agent端通过MCP客户端调用MCP服务端的工具方法，形成完整的“调用-响应”链路，同时借助Nacos实现负载均衡调度。

四、sass-to-mcp-agent 项目详情

该项目为服务调用端，负责与用户交互、发现MCP服务并发起调用，核心包含依赖配置、配置文件、核心代码三大模块，具体实现如下：

4.1 项目依赖

引入核心依赖，涵盖日志、MCP客户端、Agent框架、千问大模型（pom.xml）：



登录掘金领取礼包

更多登录后权益等你解锁

×

置如下

```
1 <dependencies>
2   <dependency>
3     <groupId>org.springframework.boot</groupId>
4     <artifactId>spring-boot-starter-logging</artifactId>
5   </dependency>
6   <dependency>
7     <groupId>org.springframework.ai</groupId>
8     <artifactId>spring-ai-starter-mcp-client-webflux</artifactId>
9   </dependency>
10  <!-- Spring AI Alibaba Agent Framework: 提供Agent核心能力 -->
11  <dependency>
12    <groupId>com.alibaba.cloud.ai</groupId>
13    <artifactId>spring-ai-alibaba-agent-framework</artifactId>
14  </dependency>
15  <!-- DashScope ChatModel 支持（千问大模型集成），若更换模型需替换对应starter -->
16  <dependency>
17    <groupId>com.alibaba.cloud.ai</groupId>
18    <artifactId>spring-ai-alibaba-starter-dashscope</artifactId>
19  </dependency>
20  <dependency>
21    <groupId>com.alibaba.cloud.ai</groupId>
22    <artifactId>spring-ai-alibaba-starter-mcp-distributed</artifactId>
23  </dependency>
24 </dependencies>
```

4.2 配置文件

通过application.yaml配置文件，设置日志、应用名称、MCP客户端、Nacos连接、千问大模型等参数，确保各组件协同工作，配置如下：

```
1 logging:
2   config: classpath:logback-spring.xml # 日志配置文件路径
3
4 spring:
5   application:
6     name: sass-to-mcp-agent # 应用名称，用于Nacos识别
7   main:
8     web-application-type: none # 非Web应用，仅启动命令行交互
9   ai:
10    dashscope:
11      api-key: 你的阿里云百炼key # 千问大模型调用密钥
12      base-url: https://dashscope.aliyuncs.com # 千问大模型请求
13    chat:
14      options:
```



登录掘金领取礼包
更多登录后权益等你解锁



```

15     model: qwen-flash # 选用的千问模型版本
16     alibaba:
17         mcp:
18             nacos:
19                 client:
20                     enabled: true # 启用Nacos作为MCP客户端注册中心
21                 sse:
22                     connections:
23                         server1:
24                             service-name: sass-to-mcp-mcp-server # Nacos中MCP服务的名称 (需与服务端一致)
25                             version: 1.0.0 # MCP服务版本 (需与服务端一致)
26                 configs:
27                     server1:
28                         namespace: public # Nacos命名空间 (默认public)
29                         server-addr: 127.0.0.1:8848 # Nacos服务地址
30                         username: nacos # Nacos登录用户名
31                         password: nacos # Nacos登录密码
32     mcp:
33         client:
34             enabled: true # 启用MCP客户端
35             name: sass-to-mcp-agent # MCP客户端名称
36             version: 0.0.1 # MCP客户端版本
37             initialized: true # 启动时初始化MCP客户端
38             request-timeout: 600s # 请求超时时间
39             type: sync # 调用类型 (同步)
40             toolcallback:
41                 enabled: true # 启用工具回调
42             root-change-notification: true # 启用根节点变更通知

```

4.3 核心代码

4.3.1 配置类

配置ReactAgent实例，集成工具回调、千问大模型、日志拦截器等，用于处理用户请求并调用MCP服务工具，代码如下：

▼ java

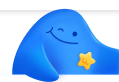


复制代码

```

1 @Configuration(proxyBeanMethods = false)
2 public class Config {
3
4     @Bean
5     public ReactAgent buildReactAgent(@Qualifier("distributedS
6         // 从nacos获取所有可用的工具回调，打印日志便于调试
7         ToolCallback[] toolCallbacks = tools.getToolCallbacks(
8         System.out.println(">>> Available tools: ");

```



登录掘金领取礼包
更多登录后权益等你解锁



ools, Cha

```

9         for (int i = 0; i < toolCallbacks.length; i++){
10             System.out.println "[" + i + "]" + toolCallbacks[i].getToolDefinition().name());
11         }
12         // 构建ReactAgent, 集成工具、日志拦截器、模型、内存存储
13         ReactAgent agent = ReactAgent.builder()
14             .tools(tools.getToolCallbacks())
15             .interceptors(new LoggingInterceptor())
16             .model(chatModel)
17             .name("test_agent")
18             .saver(new MemorySaver())
19             .build();
20         return agent;
21     }
22 }

```

4.3.2 命令行交互类

实现命令行交互功能，接收用户输入的指令，通过ReactAgent调用MCP服务，返回处理结果，支持“exit”命令退出，代码如下：

▼ java



复制代码

```

1  @Controller
2  public class CommandLineRunnerController implements CommandLineRunner {
3
4      private static final Logger logger = LoggerFactory.getLogger(CommandLineRunnerController.class);
5
6      // 注入ReactAgent实例 (由配置类创建)
7      ReactAgent agent;
8
9      public CommandLineRunnerController(ReactAgent agent) {
10         this.agent = agent;
11     }
12
13     @Override
14     public void run(String... args) throws Exception {
15         // 生成唯一对话ID, 用于标识当前对话上下文
16         RunnableConfig runnableConfig = RunnableConfig.builder()
17             .threadId(UUID.randomUUID().toString())
18             .addMetadata("user_id", "1")
19             .build();
20         // 启动控制台扫描器, 接收用户输入
21         Scanner scanner = new Scanner(System.in);
22         while (true) {
23             System.out.print("\n>>> QUESTION: ");
24             String userInput = scanner.nextLine();

```



登录掘金领取礼包
更多登录后权益等你解锁



```

25 // 输入exit,退出程序
26 if (userInput.equalsIgnoreCase("exit")) {
27     break;
28 }
29 // 空输入时,默认发送“你好”进行测试
30 if (userInput.isEmpty()) {
31     userInput = "你好";
32 }
33 String res = "";
34 try {
35     // 调用Agent处理用户输入,获取响应结果
36     AssistantMessage response = agent.call(userInput, runnableConfig);
37     res = response.getText();
38 } catch (GraphRunnerException e) {
39     // 捕获异常,打印日志并返回异常提示
40     logger.error("", e);
41     res = "系统异常";
42 }
43 // 输出Agent响应结果
44 System.out.println("\n>>> ASSISTANT: " + res);
45 }
46 // 关闭扫描器,释放资源
47 scanner.close();
48 }
49
50 }

```

五、sass-to-mcp-mcp 项目详情

该项目为服务提供端,负责将传统SASS服务封装为MCP服务,并注册到Nacos,支持多实例部署,核心包含依赖配置、配置文件、核心代码三大模块,具体实现如下:

5.1 项目依赖

引入核心依赖,涵盖Web服务、日志、MCP服务注册、MCP服务端等,确保SASS服务能正常封装为MCP服务并注册到Nacos,依赖配置如下 (pom.xml) :

```

xml
1 <dependencies>
2   <dependency>
3     <groupId>org.springframework.boot</groupId>
4     <artifactId>spring-boot-starter-web</artifactId>
5   </dependency>

```



登录掘金领取礼包
更多登录后权益等你解锁

复制代码



```

6     <dependency>
7         <groupId>org.springframework.boot</groupId>
8         <artifactId>spring-boot-starter-logging</artifactId>
9     </dependency>
10 <!-- MCP服务注册依赖，用于将MCP服务注册到Nacos -->
11 <dependency>
12     <groupId>com.alibaba.cloud.ai</groupId>
13     <artifactId>spring-ai-alibaba-starter-mcp-registry</artifactId>
14 </dependency>
15 <!-- MCP服务端依赖，支持WebMVC模式 -->
16 <dependency>
17     <groupId>org.springframework.ai</groupId>
18     <artifactId>spring-ai-starter-mcp-server-webmvc</artifactId>
19 </dependency>
20 </dependencies>

```

5.2 配置文件

通过application.yaml配置文件，设置端口、应用名称、MCP服务、Nacos注册等参数，支持多实例部署（需手动修改端口），配置如下：

▼ yaml



复制代码

```

1 logging:
2   config: classpath:logback-spring.xml # 日志配置文件路径
3 server:
4   port: 8090 # 服务端口 (多实例部署时需修改, 如8091、8092)
5
6 spring:
7   application:
8     name: sass-to-mcp-mcp # 应用名称
9   ai:
10    mcp:
11     server:
12       name: sass-to-mcp-mcp-server # MCP服务名称 (需与Agent端配置一致)
13       version: 1.0.0 # MCP服务版本 (需与Agent端配置一致)
14       type: SYNC # 调用类型 (同步)
15       instructions: "这个mcp提供订单、订单商品相关的工具与资源" # MCP服务描述
16   alibaba:
17     mcp:
18       nacos:
19         server-addr: localhost:8848 # Nacos服务地址 (与Agent端
20         namespace: public # Nacos命名空间 (默认public)
21         username: nacos # Nacos登录用户名
22         password: nacos # Nacos登录密码

```



登录掘金领取礼包
更多登录后权益等你解锁



```
23         register:
24             enabled: true # 启用Nacos服务注册 (必须开启, 否则Agent端无法发现)
```

5.3 核心代码

5.3.1 MCP服务类

通过自定义注解@McpTools标记该类为MCP服务类，将SASS服务的方法封装为MCP工具方法，供Agent端调用，代码如下：

▼ java  复制代码

```
1  @McpTools
2  public class McpService {
3      // 注入原有SASS服务的Controller, 复用原有业务逻辑
4      @Autowired
5      SassController SassController;
6
7      /**
8       * 工具方法: 获取订单详情
9       * @param param 订单详情查询参数 (需与SASS服务参数一致)
10      * @return 订单详情VO
11      */
12      @Tool(description = "获取订单详情")
13      public OrderDetailVO getOrderDetail(OrderDetailQueryParam param) {
14          // 直接调用SASS服务的方法, 实现快速封装
15          return SassController.getOrderDetail(param);
16      }
17
18      /**
19       * 工具方法: 获取分页订单列表
20       * @param param 订单列表查询参数 (含分页信息)
21       * @return 分页订单列表结果
22       */
23      @Tool(description = "获取订单列表 (分页) ")
24      public PageResult<OrderVO> getOrderList(OrderListQueryParam param) {
25          // 直接调用SASS服务的方法, 实现快速封装
26          return SassController.getOrderList(param);
27      }
28
29  }
```

说明：



登录掘金领取礼包
更多登录后权益等你解锁

×

- @McpTools: 自定义注解，用于标记MCP服务类，Spring会通过扫描机制，识别该类并注册为MCP服务。
- @Tool: 标记方法为MCP工具方法，description属性用于描述方法功能，供Agent端识别并调用。
- 依赖注入SassController，直接复用原有SASS服务的业务逻辑，无需重复开发，实现传统SASS服务向MCP服务的快速转型。

5.3.2 MCP服务扫描注册类

实现BeanFactoryPostProcessor接口，用于扫描所有带有@McpTools注解的Bean，自动注册为MCP工具回调提供者，确保Agent端能发现并调用MCP服务方法，代码如下：

▼ java



复制代码

```
1  @Component
2  public class McpToolRegister implements BeanFactoryPostProcessor {
3
4      @Override
5      public void postProcessBeanFactory(ConfigurableListableBeanFactory beanFactory) throws BeansException {
6          // 判断Bean工厂是否为BeanDefinitionRegistry, 用于注册Bean定义
7          if (beanFactory instanceof BeanDefinitionRegistry) {
8              BeanDefinitionRegistry registry = (BeanDefinitionRegistry) beanFactory;
9
10             // 获取所有带有@McpTools注解的Bean名称
11             String[] beanNames = beanFactory.getBeanNamesForAnnotation(McpTools.class);
12
13             // 遍历所有@McpTools标记的Bean, 注册为MethodToolCallbackProvider
14             for (String beanName : beanNames) {
15                 BeanDefinition bd = registry.getBeanDefinition(beanName);
16
17                 try {
18                     // 获取Bean的Class对象
19                     Class<?> beanClass = Class.forName(bd.getBeanClassName());
20
21                     // 注册MethodToolCallbackProvider, 用于提供MCP工具回调
22                     registerMethodToolCallbackProvider(registry, beanName, beanClass);
23
24                 } catch (ClassNotFoundException e) {
25                     // 捕获类加载异常, 打印堆栈信息
26                     e.printStackTrace();
27                 }
28             }
29         }
30     }
31
32     /**
```



登录掘金领取礼包
更多登录后权益等你解锁



```

33     * 注册MethodToolCallbackProvider Bean, 用于将MCP服务方法封装为工具回调
34     * @param registry Bean定义注册中心
35     * @param originalBeanName 原有@McpTools标记的Bean名称
36     * @param originalBeanClass 原有@McpTools标记的Bean的Class
37     */
38     private void registerMethodToolCallbackProvider(BeanDefinitionRegistry registry, String originalBeanName, C
39         // 生成MethodToolCallbackProvider的Bean名称, 避免与原有Bean冲突
40         String providerBeanName = "methodToolCallbackProvider_" + originalBeanName;
41
42         // 如果该Bean已注册, 则跳过, 避免重复注册
43         if (registry.containsBeanDefinition(providerBeanName)) {
44             return;
45         }
46

```



稀土掘金

首页

探索稀土掘金



创作者中心

登录 注册

```

50
51     // 通过构造函数参数, 传递MCP服务类的Class对象
52     definition.getConstructorArgumentValues().addGenericArgumentValue(Arrays.asList(originalBeanClass));
53     // 注册Bean定义, 完成MCP工具回调提供者的注册
54     registry.registerBeanDefinition(providerBeanName, definition);
55 }
56 }

```

六、项目运行流程

项目运行需遵循固定流程, 确保服务注册、发现与调用正常, 具体步骤如下:

- 启动Nacos服务:** 首先启动本地Nacos服务 (版本3.1.1) , 确保Nacos控制台可正常访问 (默认地址: <http://127.0.0.1:8848/nacos>, 用户名/密码: nacos/nacos) 。
- 启动sass-to-mcp-mcp项目 (多实例) :**
 - 启动第一个实例: 使用默认配置, 端口为8090, 启动命令无需修改端口。
 - 启动第二个实例: 手动指定端口, 示例: `java -jar sass-to-mcp-mcp.jar --server.port=8091` (可根据实际情况修改端口)
 - 启动完成后, 可在Nacos控制台“服务管理-服务列表”中查看, 且包含2个实例。



示例: `java -jar sass-to-mcp-`

`mcp.jar --server.port=8091` (可根据实际情况修改端口)

登录掘金领取礼包

更多登录后权益等你解锁

注册,

3. **启动sass-to-mcp-agent项目**：启动Agent项目，启动成功后，控制台会打印可用的MCP工具列表，同时提示">>> QUESTION: "，等待用户输入。
4. **测试服务调用与负载均衡**：在Agent控制台输入相关问题（如“获取订单详情”“获取订单列表”），Agent会自动调用MCP服务，返回处理结果；多次输入相同指令，可通过Nacos控制台或服务日志，观察请求是否在两个MCP实例间均衡分发，验证负载均衡效果。

七、运行截图

nacos mcp服务截图

NACOS

首页Nacos企业版MCP市场文档博客社区Ennacos

NACOS 3.1.1

模式 standalone

配置管理

服务管理

MCP管理new

MCP列表

Agent管理new

权限控制

MCP管理

public

创建 MCP Server导入 MCP ServerServer Name

查询

	MCP Server	支持能力	类型	版本	操作
☐	sass-to-mcp-mcp-server	TOOL	mcp-sse	1.0.0	详情 编辑 删除 禁用

删除

每页显示: 10 总数1

< 上一页1下一页 >

掘金技术社区 @ 程序员越



登录掘金领取礼包

更多登录后权益等你解锁

×

nacoss mcp 方法元数据配置截图

配置列表

历史版本

监听查询

服务管理

MCP管理

Agent管理

权限控制

命名空间

集群管理

创建配置

Data ID

已开启默认模糊查询

Group

已开启默认模糊查询

默认模糊匹配

查询

高级查询

导入配置

查询到 3 条满足要求的配置。

	Data ID	Group	格式	归属应用	操作
☐	958c44ba-59ec-4025-8d82-a57b7c8f83a2-1.0.0-mcp-tools.json	mcp-tools	JSON	sass-to-mcp-mcp-server	详情 示例代码 编辑 删除 ...
☐	958c44ba-59ec-4025-8d82-a57b7c8f83a2-mcp-versions.json	mcp-server-versions	JSON	sass-to-mcp-mcp-server	详情 示例代码 编辑 删除 ...
☐	958c44ba-59ec-4025-8d82-a57b7c8f83a2-1.0.0-mcp-server.json	mcp-server	JSON	sass-to-mcp-mcp-server	详情 示例代码 编辑 删除 ...

发起请求

Problem...

Git Sta...

Search

Console

Progre...

History

Call Hi...

AgentApplication [Java Application]

D:\jdk-25_windows-x64_bin\jdk-25.0.1\bin\javaw.exe (2026年3月10日 10:00:00)

>>> QUESTION: 订单列表

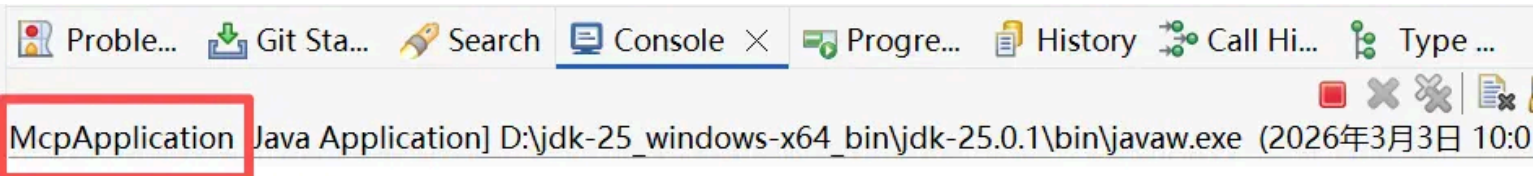
>>> ASSISTANT: 您的订单列表如下:

订单号	订单金额 (元)	订单状态	创建时间	手机号
ORD1000	199.99	已支付	2026-02-03 10:00:00	13800138000
ORD1001	209.99	已支付	2026-02-03 10:00:00	13800138000
ORD1002	219.99	已支付	2026-02-03 10:00:00	13800138000
ORD1003	229.99	已支付	2026-02-03 10:00:00	13800138000
ORD1004	239.99	已支付	2026-02-03 10:00:00	13800138000
ORD1005	249.99	已支付	2026-02-03 10:00:00	13800138000

登录掘金领取礼包

更多登录后权益等你解锁

服务1收到请求



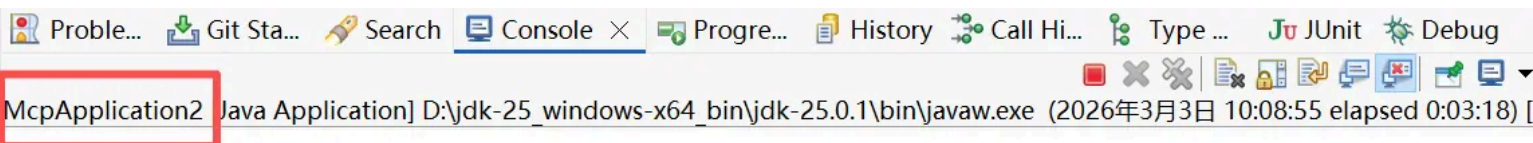
```
8280 (D:\wanzhiyue\project\example-project\spring-ai-alibaba-sass-to-mcp
profile: "default"
ngframework.ai.mcp.server.common.autoconfigure.annotations.McpServerAnno
notatedBeanRegistry' of type [org.springframework.ai.mcp.server.common.ai
i.mcp.server.annotation-scanner-org.springframework.ai.mcp.server.common
ts: []
```

```
PlatformDependent0$2 (file:/D:/m2/repository/com/alibaba/nacos/nacos-client/2.2.0/nacos-client-2.2.0.jar!classes)
Internal.PlatformDependent0$2
```

arning for 5.051)

er - 调用订单商品列表接口, 页码: 1, 每页大小: 5, 订单号: ORD100

服务2收到请求



```

PID 18908 (D:\wanzhiyue\project\example-project\spring-ai-alibaba-sass-to-mcp\sass-to-mcp)
    java.lang.NoClassDefFoundError: Could not initialize class org.springframework.ai.mcp.server.common.autoconfigure.annotations.McpServerAnnotationScanner$McpServerAnnotatedBeanRegistry' of type [org.springframework.ai.mcp.server.common.autoconfigure.annotations.McpServerAnnotationScanner$McpServerAnnotatedBeanRegistry]
    org.springframework.ai.mcp.server.annotation-scanner-org.springframework.ai.mcp.server.common.autoconfigure.annotations.McpServerAnnotationScanner$McpServerAnnotatedBeanRegistry
    objects: []

```

```
al.PlatformDependent0$2 (file:/D:/m2/repository/com/alibaba/nacos/nacos-client/3.1.0/nacos-client-3.1.0.jar!internal.PlatformDependent0$2
```

```
ss running for 4.46)
```

```
roller - 调用订单列表接口, 页码: 1, 每页大小: 10, 手机号: null, 订单号: null, 开
```

八、总结

结合实战过程中的经验，注意事项如下：



登录掘金领取礼包

更多登录后权益等你解锁

- 若新增/修改了MCP服务的方法定义（如方法参数、返回值、注解信息等），需清除Nacos中的MCP服务，否则将无法生效，甚至可能导致服务调用异常。
- Agent端（MCP调用方）必须在所有MCP服务实例启动并成功注册到Nacos后，再启动Agent项目；若提前启动Agent，可能会因无法从Nacos发现MCP服务而报错

项目源码：download.csdn.net/download/Th...

标签： Spring Spring Boot MCP 话题： 每天一个知识点

本文收录于以下专栏

Spring Boot
进阶

Spring Boot 全家桶

专栏目录

Spring Boot 是简化 Spring 应用开发的核心入门内容，包括核心注解使用、配置文件管...
8 订阅 · 37 篇文章

订阅

上一篇 接口校验神器！ Spring Boot Validati... 下一篇 Spring Boot 集成 RabbitMQ ： 8 个...

评论 8

登录 / 注册 即可发布评论！

最热 最新

Marcher丶

博主，依赖注入SassController，直接复用原有SASS服务的业务逻辑，无需重复开发，实现传统SASS服务向MCP服务的快速转型。这里是不是行不通。一般白

4月前 点赞 4

登录掘金领取礼包
更多登录后权益等你解锁

✕

...
我的例

程序员越

作者 : 为甚么运行不起来？如果项目不是

子是能运行的

4月前 点赞 回复

...



Marcher、 回复 程序员越 作者 : 我的意思是, 把controller直接依赖进来, 但是controller 一般都是最外层了, yml配置文件都在这层, 当作服务启动项目, 直接依赖 比如mybatis配置之类的无法传递吧。你的例子里 就是一个简单的controller类, 没有把他所在的 jar 模块都传递进来。或者说 你设计的初衷 是把原有的web配置还需要在 mcp server进行重新配置

4月前 1 回复

...

查看全部 4 条回复



origin_fit

源码可以分享到gitee或者github上吗 😊

4月前 点赞 1

...



程序员越 作者 : gitee.com

4月前 点赞 回复

...



Marcher、

也实现过

4月前 1 评论

...

目录

收起 ^

一、项目实现目标

二、项目运行环境

三、项目整体架构

四、sass-to-mcp-agent 项目详情

4.1 项目依赖

4.2 配置文件

4.3 核心代码

4.3.1 配置类

4.3.2 命令行交互类

五、sass-to-mcp-mcp 项目详情



登录掘金领取礼包

更多登录后权益等你解锁

×

- 5.1 项目依赖
- 5.2 配置文件
- 5.3 核心代码
 - 5.3.1 MCP服务类
 - 5.3.2 MCP服务扫描注册类

- 六、项目运行流程
- 七、运行截图
- 八、总结

搜索建议

搜索关键词

Q

- Spring AI Alibaba + Nacos 动态 MCP Server 代理方案
- 面试官：如何实现企业级MCP分布式部署？
- Spring AI Alibaba 发布企业级 MCP 分布式部署方案
- 微服务架构下的服务注册与发现：Nacos+Smoothcloud 润云 AI 算力平台实战指南
- AI 播客 + AI 博客，Nacos3.0 首个线下沙龙回顾总结（附 PPT 下载链接）
- LLM大模型智能引擎实战-SpringAI+RAG+MCP+实时搜索
- Spring AI Alibaba MCP Gateway：零代码实现存量应用转换 MCP
- SpringAI+GitHub-MCP学习记录
- SpringCloud Nacos + Ribbon 调用服务的 2 种方法！

为你推荐

- Nacos2.0的K8s服务发现生态应用及规划

阿里云云栖号4年前2.7k5评论

Java数据库
- APISIX+Dubbo+Nacos 最佳实践

阿里巴巴中间件3年前4.8k18评论

云原生Dubbo
- nacos + LoadBalancer + RestTemplate 实现远程调用负载均衡

李博帅3年前1.6k点赞评论

后端



登录掘金领取礼包

更多登录后权益等你解锁

X

Ribbon负载均衡（四）SpringCloud Ribbon负载均衡之Nacos实现

贾子杰 3年前 429 1 评论 Spring Cloud

聊聊 Nacos

AgentLee 4年前 2.0k 10 评论 后端

如何用 Nacos 构建服务网格生态

阿里云云原生 4年前 1.4k 5 评论 云原生 开源 后端

如何用 Nacos 构建服务网格生态

阿里云云栖号 4年前 635 4 评论 运维 微服务

2021升级版微服务教程6—Ribbon使用+原理+整合Nacos权重+实战优化 一篇搞定

路导 5年前 556 4 1 评论 Java

在kubernetes+istio中通过FQDN请求Nacos服务

尹安灿 3年前 883 点赞 评论 云原生

微服务开源生态报告 No.10

芋宝宝 6年前 330 点赞 评论 JavaScript

Nacos源码学习系列第1篇服务搭建-单机模式

求真极限 4年前 836 点赞 评论 架构

Spring Cloud Alibaba 入门学习笔记第二篇： Nacos注册中心+Loadbalancer负载均衡

海加尔金鹰 4年前 1.2k 1 评论 评论 Java 后端

微服务开源生态报告 No.10

阿里云云栖号 6年前 467 点赞 评论 微服务

【微服务01】Nacos + OpenFeign + LoadBalancer 的入门级用法

柿柿如意 3年前 3.1k 3 评论 后端 Java 微服务

服务通信之负载均衡器（Nacos + RestTemplate + LoadBalancer）

李博帅 3年前 229 点赞 评论 后端



登录掘金领取礼包
更多登录后权益等你解锁

